AI-Assisted Indian Tech Job Market Analysis and Candidate Matching Dashboard

Submitted in partial fulfillment of the requirements of the degree

BACHELOR OF ENGINEERING
IN INFORMATION TECHNOLOGY

By

Yash Jadhav 24101B2003

Darshan Jain 24101B2002

Chetna Padhi 24101B2005

Supervisor

Dr. Neha Kudu



Department of Information Technology
Vidyalankar Institute of Technology
Vidyalankar Educational Campus,
Wadala (E), Mumbai – 400 037
University of Mumbai
(AY 2026–2027)

Certificate

This is to certify that the Mini Project entitled **" AI-Assisted Indian Tech Job Market Analysis and Candidate Matching Dashboard "** for the Data Analytics and Visualization (BE INFT, Sem 7) is a bonafide work of **Yash Jadhav (24101B2003)**, **Darshan Jain (24101B2002)** and **Chetna Padhi (24101B2005)** submitted to the University of Mumbai in partial fulfillment of the requirement for the award of the degree of **"Bachelor of Engineering"** in **"Information Technology"** at Vidyalankar Institute of Technology, Academic Year 2026–2027.

_____

Dr. Neha Kudu
Supervisor

_____ _____

Head of Department Principal

Contents

Certificate ..... **ii**

Abstract ..... **iv**

Acknowledgement ..... **v**

List of Figures ..... **vi**

Chapter 1 Introduction **1**

1.1 Introduction 1
1.2 Motivation 1
1.3 Problem Statement & Objectives 2
1.4 Organization of the Report 3

Chapter 2 Literature Survey **4**

2.1 Survey of Existing/Similar Systems 4
2.2 Limitations of Existing/Similar Systems or Research Gap 5

Chapter 3 Proposed System **7**

3.1 Introduction 7
3.2 Details of Hardware & Software 9
3.3 Experiment and Results 10
3.4 Conclusion and Future Scope 22

References **23**

Abstract

The Indian technology job market changes rapidly, and job seekers find it difficult to understand which roles are in demand, which skills employers ask for, and which jobs they are actually eligible for. Job portals list thousands of postings but present them as long text lists. This project, "AI-Assisted Indian Tech Job Market Analysis and Candidate Matching Dashboard", applies data analytics and visualization to a dataset of 23,201 Indian tech job postings to turn this raw information into clear, explainable insights.

The system first cleans and validates the dataset, normalises inconsistent skill names through a skill taxonomy, and visualises the market by role category, city, work mode, experience tier, fresher-friendliness, disclosed salary and skill demand. Roles are ranked using a Role Opportunity Score. A candidate-job matching engine then scores a candidate profile against every posting using a weighted match score of skills, experience, role relevance, location and work mode, decides eligibility, explains each decision, ranks the most important missing skills and simulates the effect of learning new skills (what-if analysis).

The analysis backend is developed in Python using pandas, NumPy and Matplotlib in Google Colab. A LangChain agent and a four-step LangGraph workflow powered by Google Gemini convert the computed results into a natural-language career report. The results are also presented through an interactive web dashboard built with FastAPI and Chart.js.

For the demonstration candidate the system found 46 eligible jobs out of 23,201 with a best match score of 96.25%, and showed that adding Power BI and Data Analysis would increase the eligible jobs to 212. The project demonstrates how data cleaning, exploratory analysis and well-designed visualizations can support data-driven career decisions.

Keywords: Data Visualization, Data Analytics, Job Market Analysis, Exploratory Data Analysis, Skill Gap Analysis, Candidate-Job Matching, pandas, Matplotlib, Chart.js, LangChain, LangGraph, Google Gemini, FastAPI.

Acknowledgement

We would like to express our sincere gratitude to our project guide, **Dr. Neha Kudu**, for her guidance, encouragement and constructive feedback throughout this project. Her suggestions helped us shape the idea of the AI-Assisted Indian Tech Job Market Analysis and Candidate Matching Dashboard into a structured and well-documented piece of work.

We thank the Head of the Department of Information Technology, the Principal, and the faculty members of Vidyalankar Institute of Technology for providing the academic environment and resources needed to carry out this project.

We also thank each other for the shared effort, discussions and patience during data preparation, analysis, development, testing and documentation. Finally, we are grateful to our families and friends for their continuous support.

Yash Jadhav
Darshan Jain
Chetna Padhi

List of Figures

Figure 3.1: System architecture of the proposed system 8
Figure 3.2: Job postings by role category 12
Figure 3.3: Top 15 cities by job postings 13
Figure 3.4: Job distribution by work mode 13
Figure 3.5: Job distribution by experience tier 14
Figure 3.6: Fresher-friendly jobs by role (percentage) 14
Figure 3.7: Average match score before and after skill addition 16
Figure 3.8: Eligible jobs before and after skill addition 16
Figure 3.9: LangGraph career-advisor workflow 17
Figure 3.10: Market Overview tab – KPI cards, postings by role and Role Opportunity Score 18
Figure 3.11: Market Overview tab – cities, core skills, salary by role and fresher share 18
Figure 3.12: Job Matcher tab – candidate profile, summary indicators and fit breakdown 19
Figure 3.13: Job card with score breakdown, explanation and matched/missing skills 19
Figure 3.14: Skill Gaps & What-If tab 20
Figure 3.15: AI Advisor tab – four-step workflow and career report 20
Figure 3.16: Explore Jobs tab – searchable and filterable job table 20

Chapter 1
Introduction

1.1 Introduction

Data visualization is an important part of data analysis because it represents complex information in a graphical and easily understandable format. Bar charts, doughnut charts and score cards help users compare categories, identify patterns and make informed decisions. When a dataset contains tens of thousands of records, visualization becomes the most practical way to understand it.

The technology job market in India is a good example of such data. Every day thousands of postings are published for roles such as Data Scientist, Data Analyst, Business Analyst, Machine Learning Engineer, Data Engineer and Python Developer. Each posting lists a location, an experience range, a work mode, a set of required skills and sometimes a salary. Reading these postings one by one does not tell a job seeker how the market is structured or where they fit in it.

The mini-project "AI-Assisted Indian Tech Job Market Analysis and Candidate Matching Dashboard" uses a dataset of 23,201 job postings with 32 attributes, collected from 6,993 companies across 198 cities. The data is cleaned, its skill lists are standardised, and the market is summarised through charts of role demand, city distribution, work mode, experience level, fresher-friendliness, disclosed salary and skill demand.

The project then moves from the market level to the individual level. A candidate enters their skills, experience, preferred city, preferred work mode and target role, and the system scores the candidate against every posting, explains why each job is or is not suitable, ranks the missing skills that matter most and simulates how the opportunities would change after learning new skills. A Google Gemini model, used through LangChain and LangGraph, writes these results as a readable career report.

The analysis is developed in Python with pandas, NumPy and Matplotlib in Google Colab, and the results are presented through an interactive web dashboard built with FastAPI and Chart.js. The project demonstrates how data analytics and visualization can transform a large, inconsistent dataset into meaningful and explainable career guidance.

1.2 Motivation

The motivation behind the project is to make the Indian tech job market easier to understand for students and early-career professionals using data analytics and visualization. Job seekers usually decide which role to target and which skill to learn next based on opinions, advertisements or a few postings they happen to read, rather than on evidence from the market as a whole.

The project is motivated by the following factors:

1. Understand the Market at a Glance: To summarise thousands of job postings into a few clear charts showing demand by role, city, work mode and experience level.

2. Make Salary and Skill Information Visible: To show which roles pay more and which skills employers ask for most, using only the data that is actually disclosed.

3. Know Where a Candidate Stands: To show a candidate exactly which jobs they are eligible for and why they are not eligible for the others.

4. Guide Skill Development: To identify the missing skills that would open up the most new opportunities and to measure their effect before the candidate invests time in learning them.

5. Explore AI with Data Analytics: To understand how large language models such as Google Gemini, combined with LangChain and LangGraph, can explain analytical results in natural language.

6. Improve User Interaction: To provide an easy-to-use dashboard where users can enter a profile, explore jobs and see results without writing any code.

Overall, the project aims to bridge the gap between raw job-postings data and practical career decisions, making the job market easier, faster and more transparent to understand.

1.3 Problem Statement & Objectives

1.3.1 Problem Statement

Job portals present technology jobs as long lists of text postings. A job seeker cannot easily see how demand is distributed across roles and cities, how much each role pays, how many jobs are open to freshers, or which skills are requested most often. Only a small fraction of postings disclose a salary, skills are written inconsistently (for example “PowerBI”, “Power-BI” and “MS Power BI”), and generic words such as “communication” or “analytical” are mixed with real technical skills.

At the individual level, a candidate does not know which of the thousands of postings they actually meet the requirements for, why they are rejected by the others, or which skill would make the biggest difference if learned next. Existing portals rank jobs by keyword relevance but do not explain the match or show the effect of improving a profile.

The project “AI-Assisted Indian Tech Job Market Analysis and Candidate Matching Dashboard” addresses this problem by cleaning and standardising the job-postings data, visualising the market, and providing an explainable candidate-job matching engine with skill-gap analysis, what-if simulation, an AI career report and an interactive dashboard.

1.3.2 Objectives

The main objectives of the project are:

1. Data Preparation: To load, validate and clean a dataset of 23,201 job postings, checking duplicates and missing values and parsing experience requirements.

2. Skill Standardisation: To normalise skill names using an alias table and classify them as technical, business, generic or other skills.

3. Market Visualization: To visualise job postings by role category, top cities, work mode, experience tier and fresher-friendliness.

4. Salary and Skill-Demand Analysis: To analyse disclosed salaries by role and identify the most demanded skills in the market and in each role.

5. Role Opportunity Scoring: To rank roles using a weighted score of demand, salary, fresher-friendliness and remote/hybrid flexibility.

6. Candidate-Job Matching: To score a candidate against every posting, decide eligibility and explain every decision with matched and missing skills.

7. Skill-Gap and What-If Analysis: To rank priority missing skills and measure the increase in eligible jobs after adding new skills.

8. AI Career Report: To generate a grounded natural-language report using LangChain, LangGraph and Google Gemini.

9. Interactive Dashboard: To present all results through a web dashboard built with FastAPI and Chart.js.

The overall objective is to transform raw job-postings data into understandable visual and textual insights that support data-driven career decisions.

1.4 Organization of the Report

This report is organized into chapters that provide a systematic explanation of the project "AI-Assisted Indian Tech Job Market Analysis and Candidate Matching Dashboard." Each chapter focuses on a specific aspect of the project.

- Chapter 1 – Introduction: Introduces the project, its background, motivation, problem statement, objectives and the organization of the report.

- Chapter 2 – Literature Survey: Discusses existing job portals, salary platforms, labour-market reports and resume-matching tools, and identifies the research gap.

- Chapter 3 – Proposed System: Explains the proposed system and its architecture, the hardware and software used, the experiments performed with their results, and the conclusion and future scope.

- References: Lists the books, research papers, documentation and online resources referred to during the project.

This organization ensures a clear and structured presentation of the project's concepts, design, implementation and outcomes.

Chapter 2 Literature Survey

2.1 Survey of Existing/Similar Systems

Job-market analysis and job matching combine data collection, data analytics, visualization and, increasingly, Artificial Intelligence. Several platforms and tools already help job seekers search for jobs, compare salaries and match their resumes to postings.

The following existing systems were studied for this project:

1. Naukri.com (Job Portal)

Naukri.com is one of the largest job portals in India. Users can search postings and filter them by keyword, location, experience, salary range and company. It is effective for finding individual jobs, but the results are presented as lists rather than as an analysis of the market as a whole.

2. LinkedIn Jobs

LinkedIn Jobs combines job postings with professional profiles. It recommends jobs based on a member's profile and activity and can indicate how a member's listed skills relate to the skills listed in a posting. The detailed reasoning behind recommendations is not fully visible to the user.

3. Glassdoor (Salary and Company Insights)

Glassdoor provides company reviews and salary information contributed by employees. It helps users compare pay for a job title across companies and locations, but the information depends on what users choose to report.

4. Indeed Hiring Lab (Labour-Market Research)

Indeed Hiring Lab publishes research and reports on hiring trends based on job-postings data. These reports describe the market at an aggregate level, but they are not personalised and do not show where an individual candidate fits.

5. Jobscan (Resume–Job Description Matching)

Jobscan compares a resume with a single job description and reports a keyword match rate with suggestions for missing keywords. It works on one posting at a time and does not analyse the wider market or simulate the effect of learning a skill across many jobs.

Comparison of Existing Systems

System	Primary Function	Main Capability
Naukri.com	Job search portal	Search and filter postings by location, experience and salary

System	Primary Function	Main Capability
LinkedIn Jobs	Professional network with jobs	Profile-based job recommendations
Glassdoor	Salary and company insights	Crowd-sourced salary and review information
Indeed Hiring Lab	Labour-market research	Aggregate reports on hiring trends
Jobscan	Resume matching tool	Keyword match rate against one job description

2.2 Limitations of Existing/Similar Systems or Research Gap

Although existing systems help job seekers in different ways, certain limitations remain.

1. List-Based Presentation: Job portals mainly show postings as lists; they rarely visualise how demand is distributed across roles, cities, work modes and experience levels.

2. Limited Explainability: Recommendation systems seldom explain why a job is recommended or why a candidate is not suitable for it.

3. One Posting at a Time: Resume-matching tools compare a resume with a single job description instead of evaluating the candidate against the whole market.

4. No What-If Analysis: Existing tools do not measure how many additional jobs a candidate would qualify for after learning a particular skill.

5. Inconsistent Skill Data: Skill names in postings are written in many forms, and generic words are mixed with technical skills, which distorts simple keyword counts.

6. Incomplete Salary Information: Salary is disclosed in only a small share of postings, so salary comparisons must be made carefully and transparently.

Research Gap

The study of existing systems highlights the need for an integrated, explainable and visual platform that analyses the job market as a whole and also evaluates an individual candidate against it.

The proposed project addresses this gap by providing the following functionalities:

Cleaning and standardising a large job-postings dataset, including skill normalisation.

Visualising the market by role, city, work mode, experience, fresher-friendliness, salary and skill demand.

Ranking roles with a transparent Role Opportunity Score.

Scoring a candidate against every posting with an explanation, matched skills and missing skills.

Ranking skill gaps and simulating the effect of learning new skills.

Generating an AI career report and presenting everything in an interactive dashboard.

Thus, the proposed system combines market analysis and personalised, explainable guidance within a single interface.

Chapter 3
Proposed System

3.1 Introduction

The proposed system, “AI-Assisted Indian Tech Job Market Analysis and Candidate Matching Dashboard,” is a data analytics and visualization application that analyses the Indian technology job market and gives each candidate an explainable view of their opportunities. It reduces the need for users to read thousands of postings by summarising the market visually and by evaluating the candidate's profile against every posting automatically.

The system processes a CSV dataset of job postings through a cleaning and feature-engineering pipeline, an analytics and matching engine, an AI advisor and a web dashboard. It is developed using Python, pandas, NumPy, Matplotlib, LangChain, LangGraph, Google Gemini, FastAPI and Chart.js, and provides the following functionalities:

1. Market Analysis: Charts and summary indicators of job postings by role, city, work mode, experience tier, fresher-friendliness, disclosed salary and skill demand, together with a Role Opportunity Score.

2. Candidate-Job Matching: A weighted match score (50% skills, 25% experience, 10% role relevance, 10% location, 5% work mode), eligibility rules, fit category, application priority and an explanation for every job.

3. Skill-Gap Analysis: A ranked list of missing core skills and the number of additional jobs each skill would unlock.

4. What-If Simulation: A before-and-after comparison of eligible jobs and average match score after adding new skills.

5. AI Career Advisor: A four-step LangGraph workflow (match → recommendation → what-if → explanation) in which Google Gemini writes a career report grounded in the computed results.

6. Interactive Dashboard: A web interface with Market Overview, Job Matcher, Skill Gaps & What-If, AI Advisor, Explore Jobs and Dataset tabs.

The match score measures how well a profile fits a posting; it is not a probability of being hired, because the dataset contains no information about hiring outcomes. The system states this wherever scores are shown.

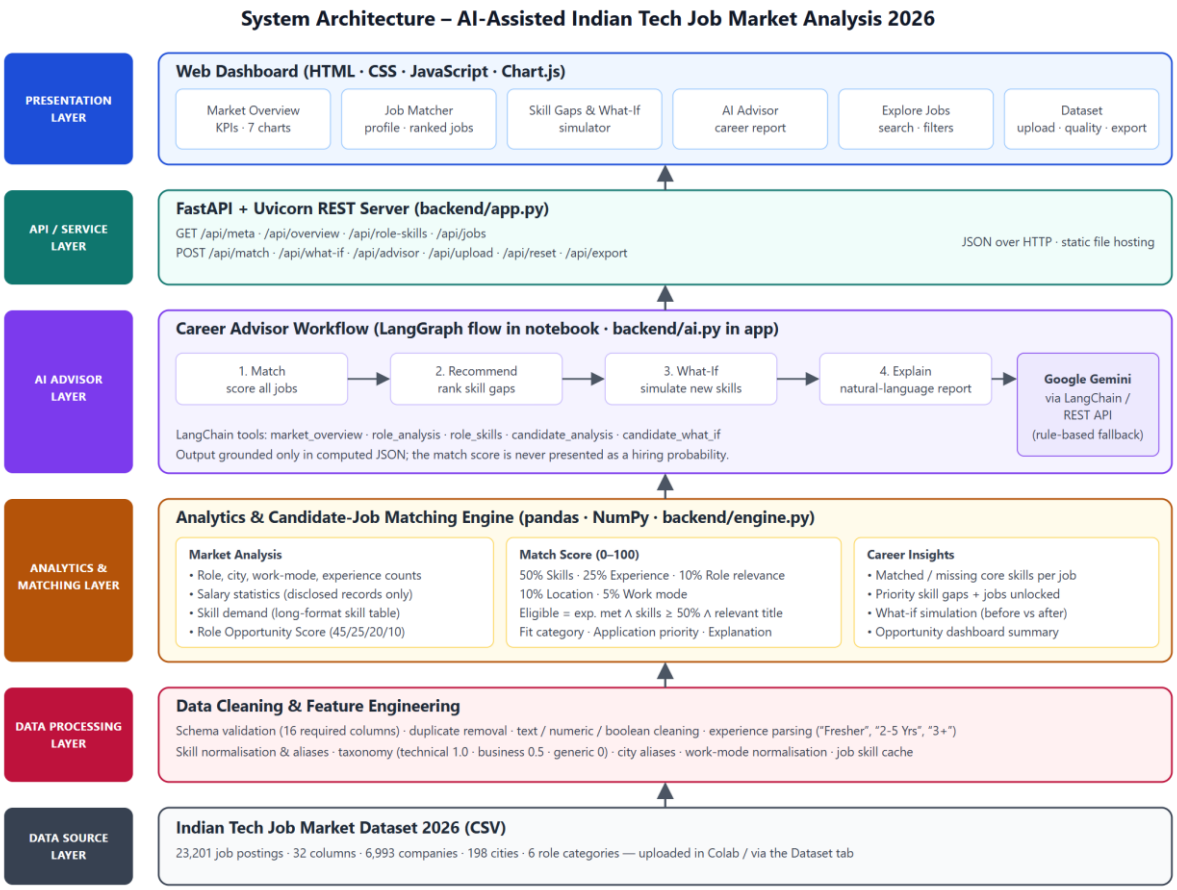


Figure 3.1: System architecture of the proposed system

As shown in Figure 3.1, data flows upwards from the dataset through the processing, analytics and AI layers to the dashboard. The data processing layer validates 16 required columns, removes duplicates, cleans text, numeric and boolean columns, parses experience text and normalises skills. The analytics layer computes the market statistics, the opportunity score and the candidate match scores. The AI layer explains the results, the FastAPI layer serves them as JSON, and the dashboard presents them as interactive charts and job cards.

3.2 Details of Hardware & Software

The proposed system uses a combination of hardware resources and software technologies to process the job-postings dataset, perform the analysis and present the results visually.

3.2.1 Hardware Requirements

The following hardware components are required to develop and execute the proposed system:

Hardware Component	Requirement / Description
Processor	Multi-core Intel/AMD processor
RAM	Minimum 8 GB recommended
Storage	Sufficient free storage for the dataset, libraries and exported files
GPU	Not required (the analysis runs on CPU)
Display	Standard monitor or laptop display
Input Device	Keyboard and mouse/trackpad
Internet Connection	Required for Google Colab, the Gemini API and the Chart.js library

Note: The analysis uses vectorised NumPy operations and does not require GPU acceleration. A candidate is scored against all job postings in a fraction of a second on a standard laptop.

3.2.2 Software Requirements

The following software tools, frameworks and libraries are used in the project:

Software / Technology	Purpose
Python 3.12	Primary programming language
Google Colab / Jupyter Notebook	Developing and running the analysis notebook
pandas 3.0	Data loading, cleaning, grouping and aggregation
NumPy 2.x	Vectorised scoring of candidates against all jobs
Matplotlib / Seaborn	Static charts in the analysis notebook
LangChain	Wrapping analysis functions as tools for the AI agent
LangGraph	Four-step career-advisor workflow
Google Gemini API	Large language model for the career report
FastAPI 0.162 / Uvicorn 0.54	REST API and web server for the dashboard
HTML, CSS, JavaScript	Dashboard layout, forms and interaction

Software / Technology	Purpose
Chart.js 4.4	Interactive bar and doughnut charts
Visual Studio Code, Google Chrome	Writing code and testing the dashboard

3.2.3 Technologies Used

1. Python: Used as the main programming language for data cleaning, analysis, the matching engine and the web backend.

2. pandas: Used to load the CSV file, clean columns, group and aggregate data, and build the long-format skill table of 176,067 skill-job records.

3. NumPy: Used to compute the match score of a candidate against every job at once instead of looping over rows.

4. Matplotlib: Used to draw sorted bar charts of roles, cities, work modes, experience tiers, fresher-friendliness and what-if results in the notebook.

5. LangChain and LangGraph: Used to expose the analysis functions as tools and to run the match, recommendation, what-if and explanation steps in order.

6. Google Gemini: Used to write the career report from the computed analysis; a rule-based advisor is used when no API key is available.

7. FastAPI and Chart.js: Used to serve the analysis results through REST endpoints and to draw the interactive charts in the dashboard.

3.2.4 Execution Environment

The project runs in two environments:

Google Colab: The analysis notebook (DAVmp.ipynb) is executed in Google Colab. The dataset is uploaded through the notebook and the Gemini API key is entered securely at run time.

Local Web Server: The dashboard is started with Uvicorn on http://localhost:8000. It loads the dataset from the data folder or from a CSV uploaded through the Dataset tab.

Web Browser: The dashboard is used in a standard web browser and adapts to desktop and mobile screen widths.

3.3 Experiment and Results

3.3.1 Experiment

The project was developed and tested to examine how effectively data analytics and visualization can describe the Indian tech job market and guide an individual candidate. The experiments involved data preparation, exploratory visualization, salary and skill-demand analysis, role scoring, candidate matching, skill-gap and what-if analysis, AI-generated explanation and the evaluation of the web dashboard.

The experiments were carried out on the Indian Tech Job Market Dataset 2026, which contains 23,201 job postings with 32 columns, using a demonstration candidate with the skills Python, SQL, Excel and Pandas, 1 year of experience, a preference for Mumbai and remote work, and the target role Data Analyst.

Experimental Procedure

The experiments were carried out in the following stages:

1. Dataset Preparation: The CSV file was uploaded in Google Colab and validated against 16 required columns. Duplicate rows and job IDs were checked, and text, numeric and boolean columns were cleaned.

2. Feature Engineering: The minimum experience was parsed from text such as “Fresher”, “2-5 Yrs” and “3+”, and the columns salary_available and remote_or_hybrid were derived.

3. Skill Taxonomy: Skill lists were split, normalised through an alias table and classified as technical (weight 1.0), business (weight 0.5), generic or other (weight 0).

4. Exploratory Visualization: Bar charts were drawn for postings by role, top 15 cities, work mode, experience tier and fresher-friendly percentage by role.

5. Salary, Skill and Role Analysis: Disclosed salaries were summarised by role, skills were ranked by demand, and roles were scored using the Role Opportunity Score (45% demand, 25% salary, 20% fresher rate, 10% flexibility).

6. Candidate Matching: The demonstration candidate was scored against all postings, and eligibility, fit category, application priority and explanations were generated.

7. Skill-Gap and What-If Analysis: Missing skills were ranked, and Power BI and Data Analysis were added to the profile to measure the change.

8. AI Explanation and Dashboard Evaluation: The LangChain agent and LangGraph workflow generated reports with Gemini, and the web dashboard was tested tab by tab.

3.3.2 Experiments Performed

Experiment	Description
Experiment 1: Data Quality and Cleaning	Validated columns, checked duplicates and missing values, and cleaned the dataset.
Experiment 2: Market Visualization	Visualised postings by role, city, work mode, experience tier and fresher-friendliness.
Experiment 3: Salary and Skill Demand	Summarised disclosed salaries by role and ranked the most demanded skills.
Experiment 4: Role Opportunity Score	Ranked the six roles using demand, salary, fresher rate and flexibility.
Experiment 5: Candidate-Job Matching	Scored the demonstration candidate against all 23,201 postings with explanations.
Experiment 6: Skill Gaps and What-If	Ranked missing skills and measured the effect of adding Power BI and Data Analysis.

Experiment	Description
Experiment 7: AI Career Advisor	Generated career reports with LangChain, LangGraph and Google Gemini.
Experiment 8: Interactive Dashboard	Presented and tested all results in the FastAPI and Chart.js web dashboard.

3.3.3 Results and Discussion

The experiments demonstrate how data analytics and visualization convert a large job-postings dataset into clear market insights and personalised guidance. The results below are taken from the notebook run on the full dataset in Google Colab.

Data Quality and Cleaning Results

The dataset loaded with 23,201 rows and 32 columns, and all 16 required columns were present. The quality check found 0 duplicate rows, 0 duplicate job IDs and only 53 missing job URLs. After cleaning, the dataset contained 23,201 records from 6,993 companies, with 13,128 unique job titles, 198 cities and 6 role categories. Only 2,766 postings (11.9%) disclosed a salary, 4,182 were fresher-friendly and 4,500 were remote or hybrid.

Market Visualization Results

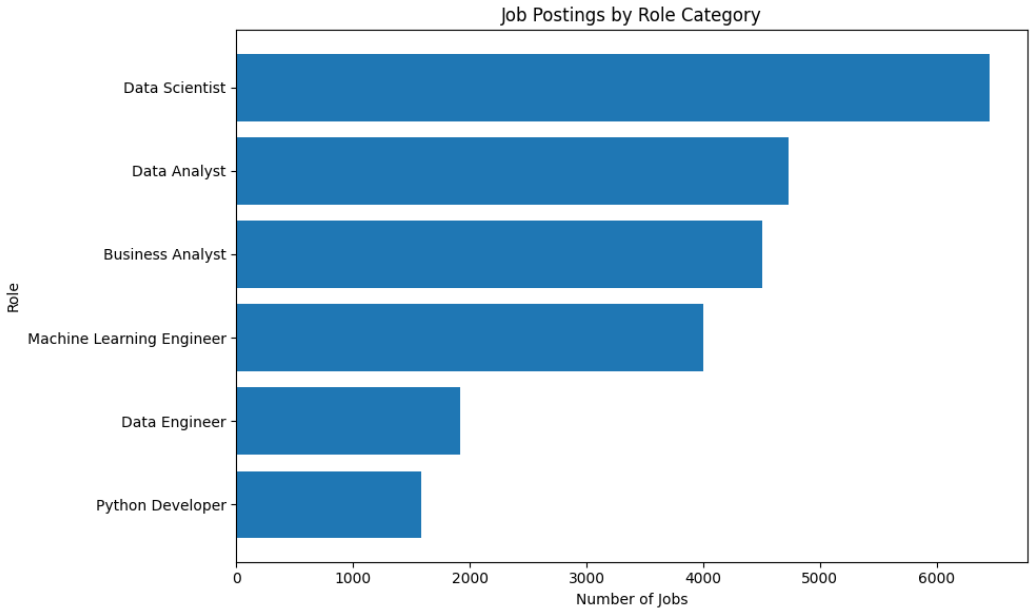


Figure 3.2: Job postings by role category

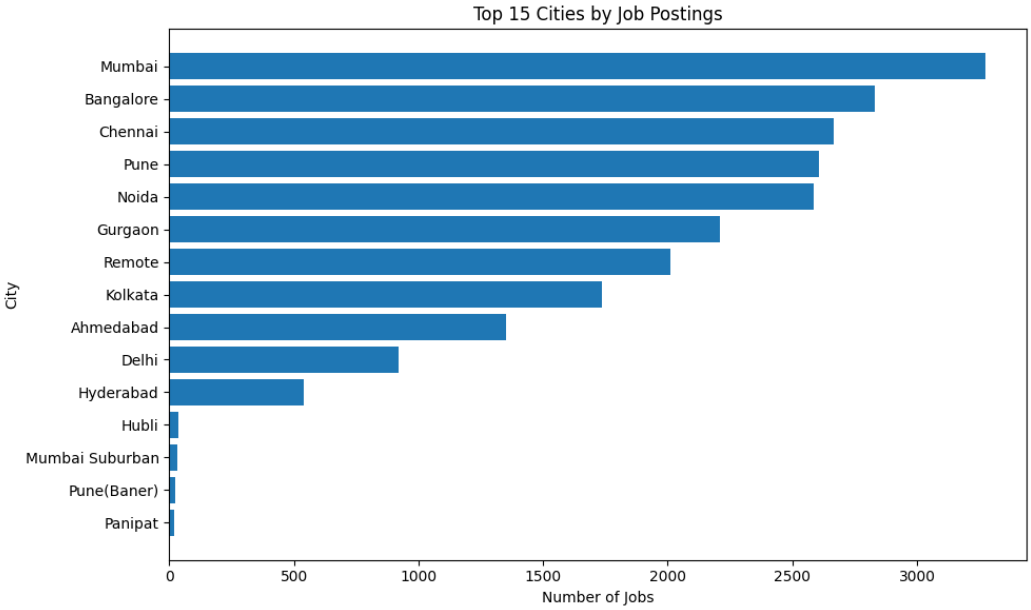


Figure 3.3: Top 15 cities by job postings

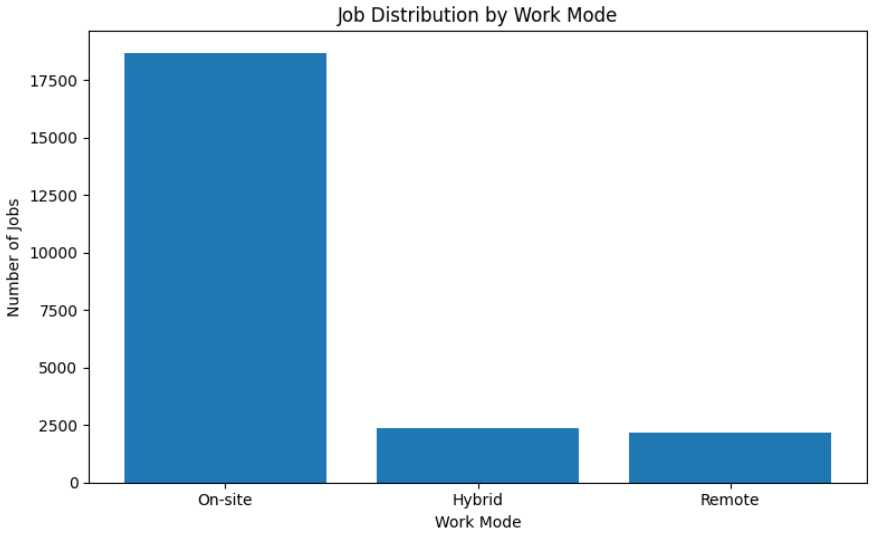


Figure 3.4: Job distribution by work mode

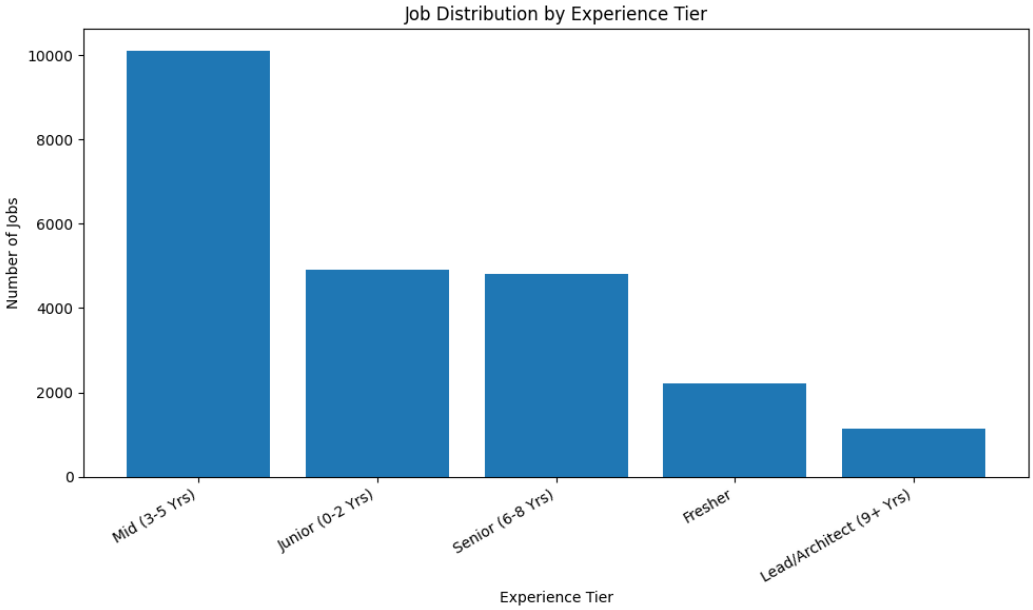


Figure 3.5: Job distribution by experience tier

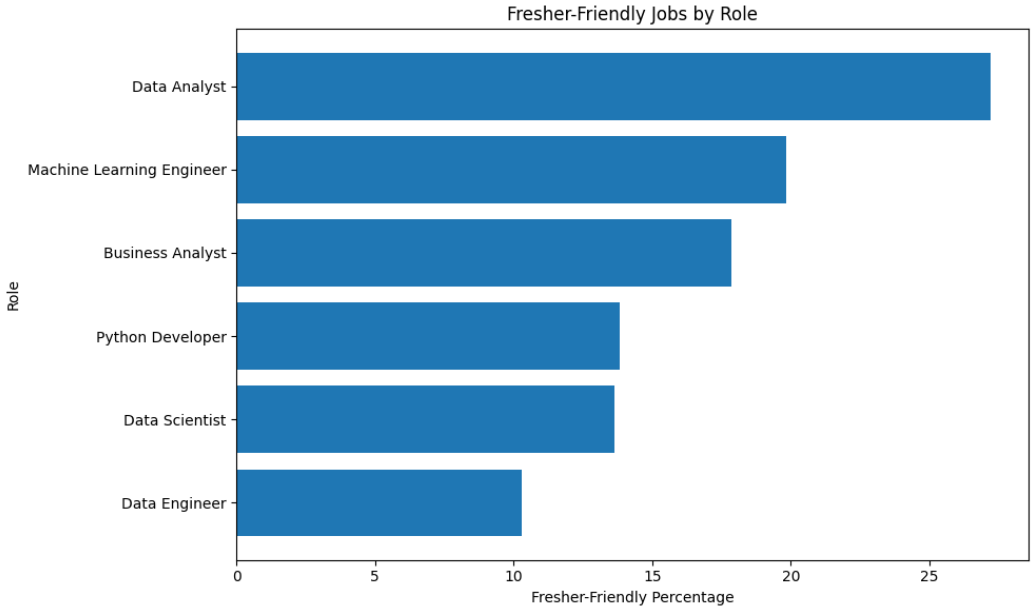


Figure 3.6: Fresher-friendly jobs by role (percentage)

Data Scientist is the largest role category with 6,455 postings, followed by Data Analyst (4,729), Business Analyst (4,505) and Machine Learning Engineer (4,004). Postings are concentrated in a few large technology cities, most jobs are on-site, and Data Analyst is the most fresher-friendly role at 27%.

Salary and Skill-Demand Results

Salary statistics were computed only for the 2,766 postings with a disclosed salary. The average disclosed salary was 15.33 LPA.

Role	Postings with Salary	Mean Salary (LPA)	Median Salary (LPA)
Data Scientist	756	18.61	17.5

Role	Postings with Salary	Mean Salary (LPA)	Median Salary (LPA)
Data Engineer	215	17.85	17.0
Machine Learning Engineer	473	16.29	14.0
Python Developer	222	15.72	15.0
Business Analyst	460	14.14	13.5
Data Analyst	640	10.62	9.5

The skill table contained 176,067 skill-job records. The most demanded skills were Python (4,270 postings, 18.40%), data analysis (3,776, 16.28%), machine learning (3,630, 15.65%), SQL (3,020, 13.02%) and artificial intelligence (2,729, 11.76%). Generic terms such as "analytical" also ranked high (2,427 postings), which confirms the need for the skill taxonomy so that such words do not distort the skill match.

Role Opportunity Score Results

Role	Jobs	Avg Salary (LPA)	Fresher Rate	Opportunity Score
Data Scientist	6,455	18.61	0.14	75.07
Machine Learning Engineer	4,004	16.29	0.20	46.59
Business Analyst	4,505	14.14	0.18	42.84
Data Analyst	4,729	10.62	0.27	35.90
Data Engineer	1,922	17.85	0.10	30.22
Python Developer	1,586	15.72	0.14	20.33

Data Scientist has the highest opportunity score because it leads in both demand and salary. Data Analyst has the highest fresher rate but a lower salary, so it ranks fourth.

Candidate-Job Matching Results

The demonstration candidate was scored against all 23,201 postings. The candidate was eligible for 46 jobs (0.2%), with 54 Excellent Fit, 332 Good Fit, 934 Potential Fit, 4,056 Low Fit and 17,825 Very Low Fit postings and an average match score of 22.05. The best matches, such as "Data Analyst - US Shift" at Morningstar and "Lead Data Analyst" at Tech Mahindra, scored 96.25%, with 100% skill match, 100% experience match and 100% role relevance. Each result was accompanied by an explanation, for example: "Eligible because the candidate has 100.0% core skill match, meets the experience requirement, and the job title is relevant to the target role."

Skill-Gap and What-If Results

The priority skill gaps for the candidate were data analysis, power bi, machine learning, business analysis, data visualization, tableau, business analyst, market research, automation and project management.

Evaluation Metric	Before	After adding Power BI and Data Analysis
Eligible Jobs	46	212 (+166)
Average Match Score	22.05	27.28 (+5.23)

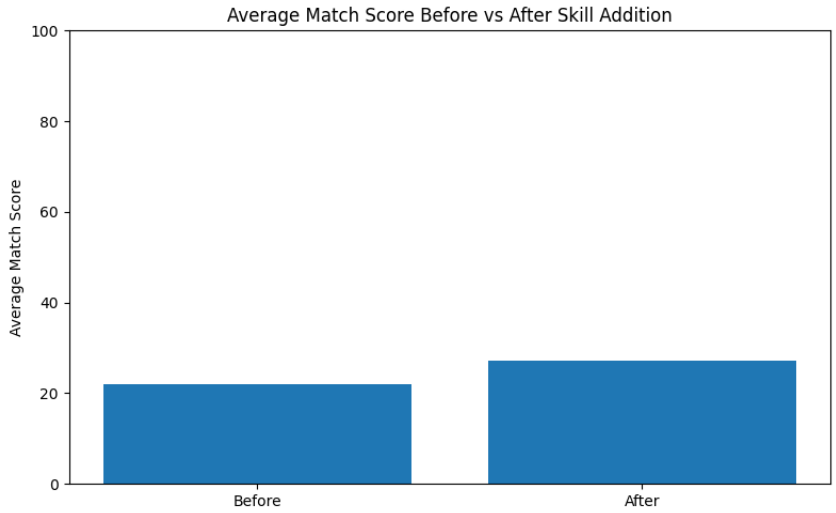


Figure 3.7: Average match score before and after skill addition

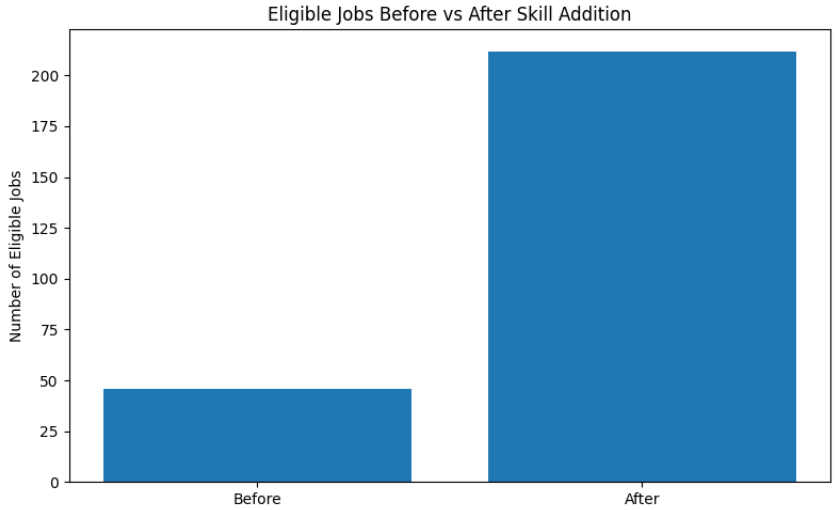


Figure 3.8: Eligible jobs before and after skill addition

Adding only two skills increased the number of eligible jobs about 4.6 times, which shows how the what-if analysis turns a skill recommendation into a measurable result.

AI Career Advisor Results

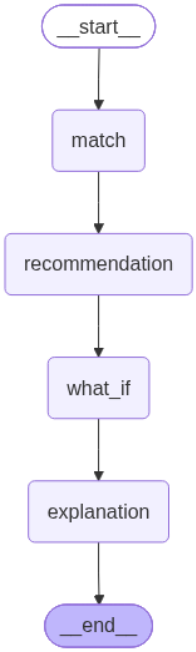


Figure 3.9: LangGraph career-advisor workflow

The Gemini connection was successful and both the LangChain agent and the LangGraph workflow were created. The generated reports used the computed values correctly (23,201 jobs analysed, 46 eligible, average match score 22.05, best match 96.25%) and stated that the match score is not a hiring probability. The LangGraph workflow tested three recommended skills (Data Analysis, Power BI and Machine Learning) and reported an increase from 46 to 228 eligible jobs and from 22.05 to 30.08 in the average match score. It recommended learning Power BI or Tableau and being more flexible about work mode.

Interactive Dashboard Results

The web dashboard presents the same analysis interactively. The screenshots below were captured on a demonstration file with the same 32-column structure, because the full CSV was not stored on the development machine; when the full dataset is uploaded through the Dataset tab, the same screens show the values reported above.

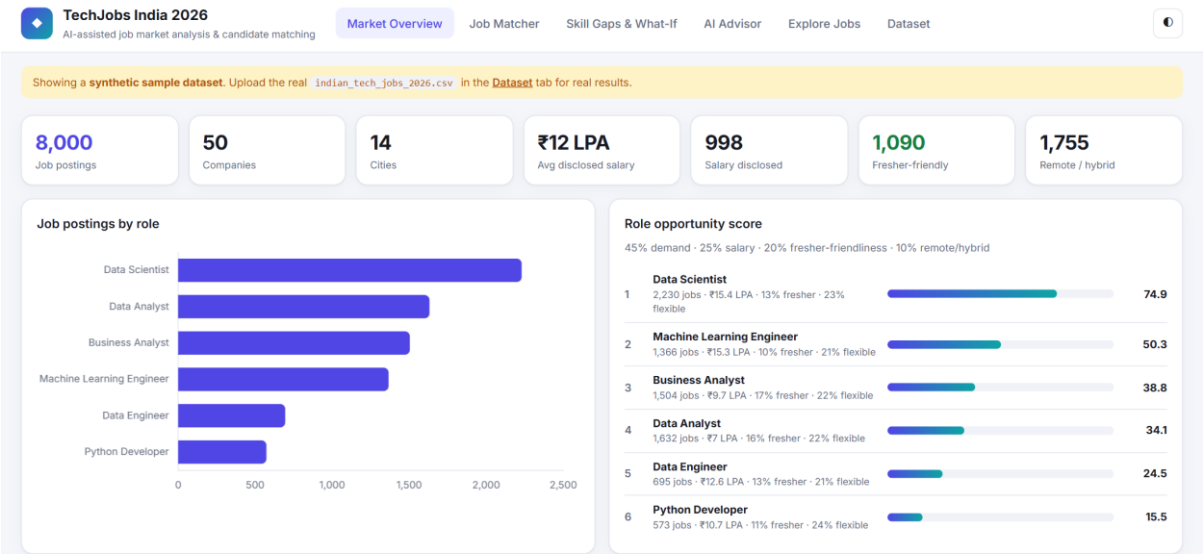


Figure 3.10: Market Overview tab – KPI cards, postings by role and Role Opportunity Score

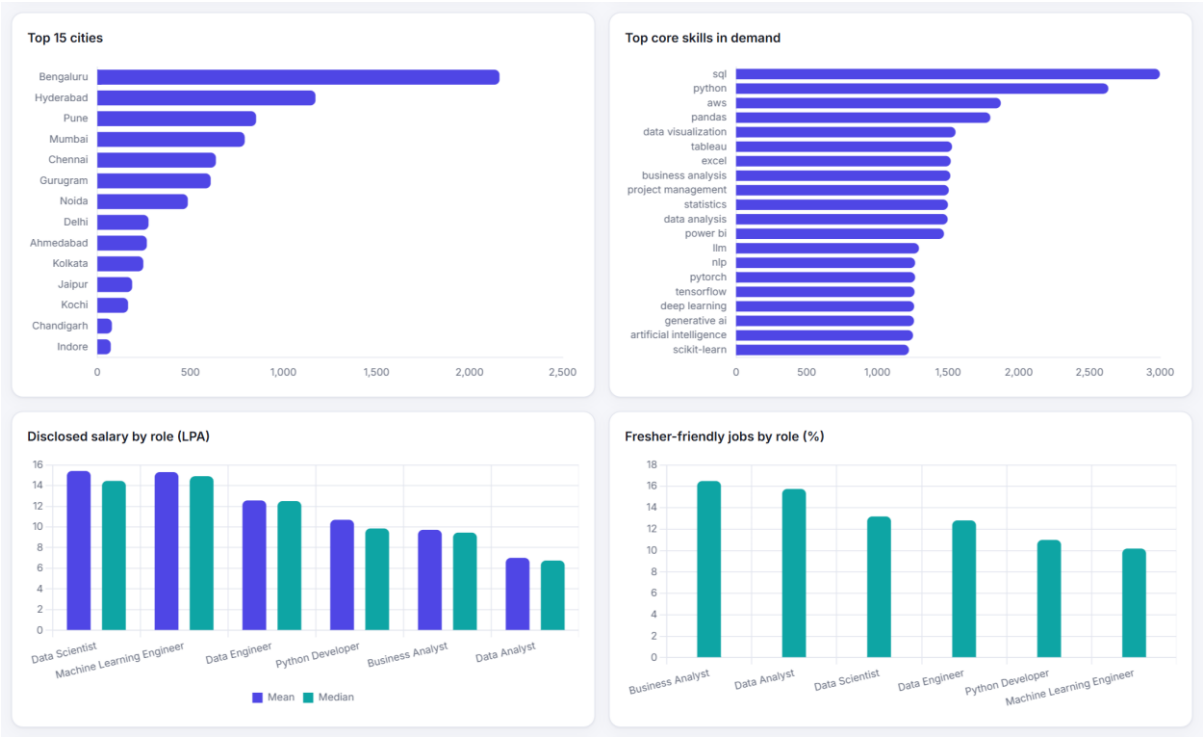


Figure 3.11: Market Overview tab – cities, core skills, salary by role and fresher share

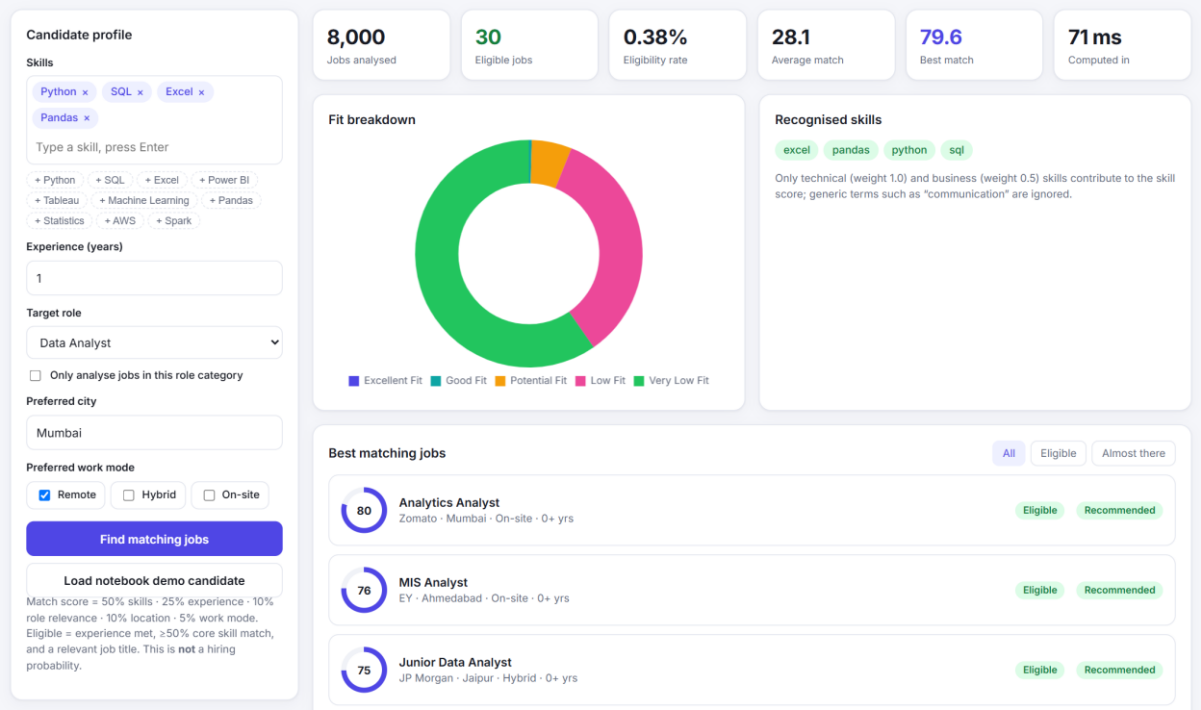


Figure 3.12: Job Matcher tab – candidate profile, summary indicators and fit breakdown

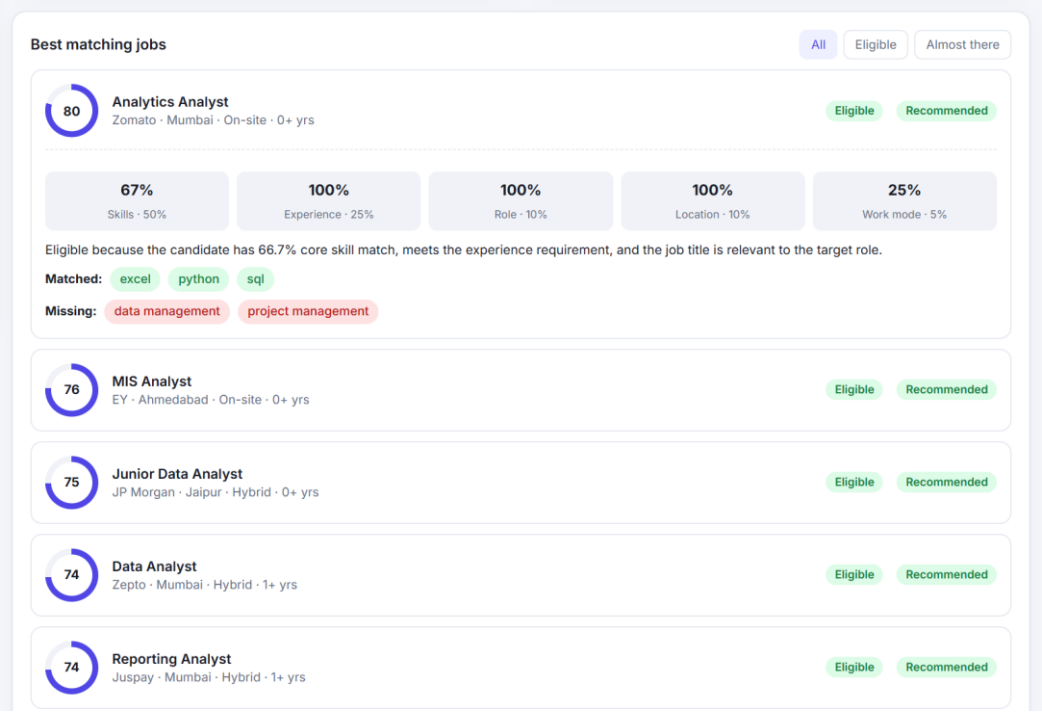


Figure 3.13: Job card with score breakdown, explanation and matched/missing skills

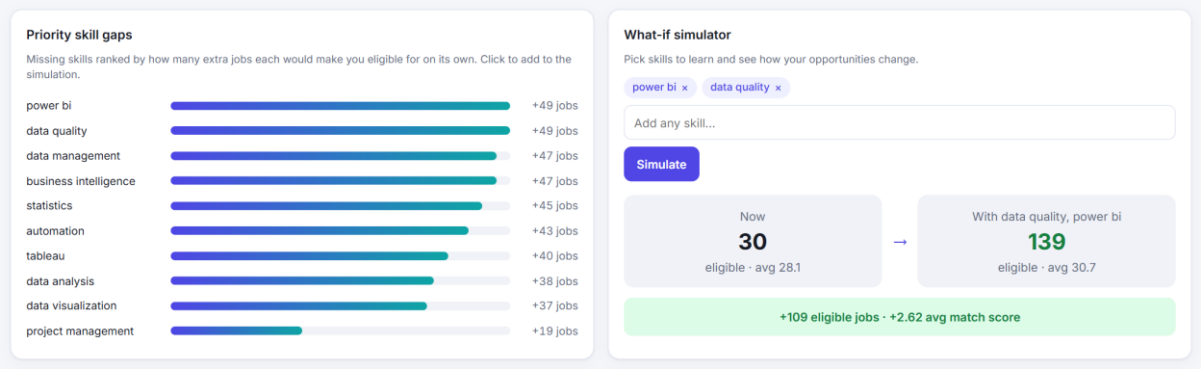


Figure 3.14: Skill Gaps & What-If tab

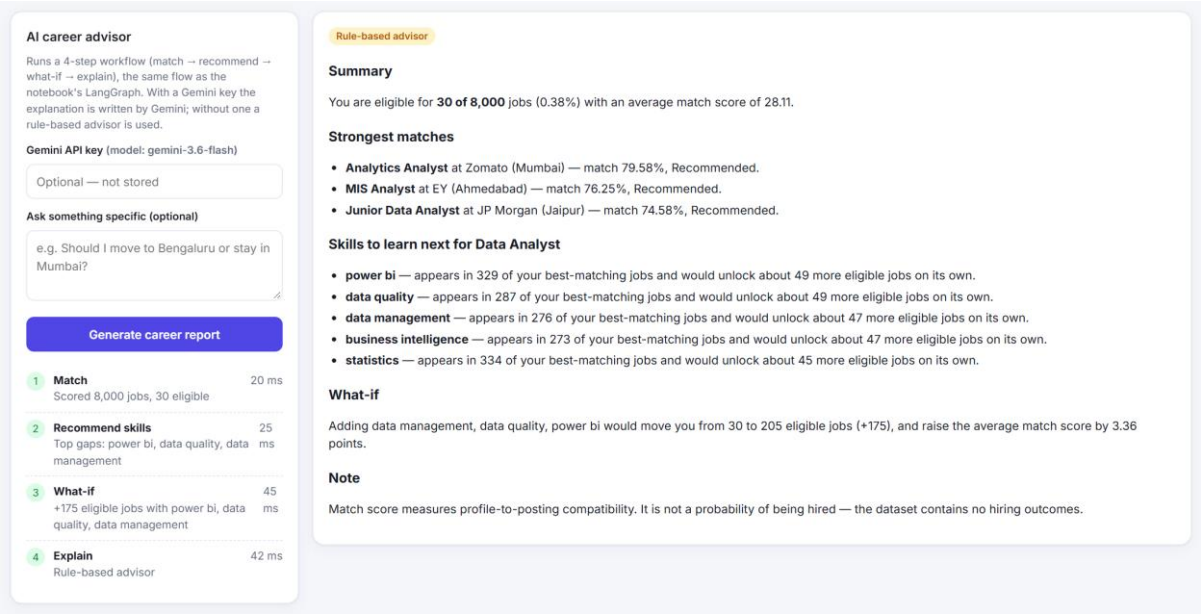


Figure 3.15: AI Advisor tab – four-step workflow and career report

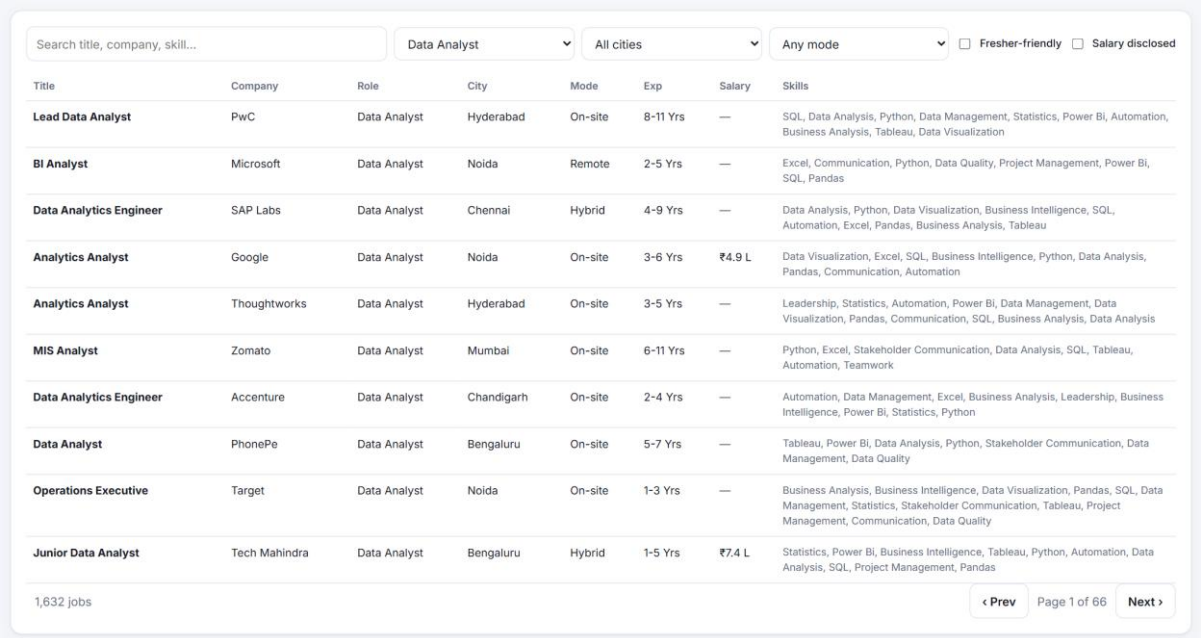


Figure 3.16: Explore Jobs tab – searchable and filterable job table

The dashboard scores a profile against every posting in well under a second and updates all indicators, charts and job cards together. Every job card shows the five score components, so the match score is never a black box.

3.3.4 Evaluation of the Scoring Model

The project uses the following scoring components and rules to evaluate a candidate against a job:

Component / Rule	Purpose
Skill Match (50%)	Weighted share of the job's core skills that the candidate has (technical 1.0, business 0.5).
Experience Match (25%)	100% if the minimum experience is met; otherwise proportional to the candidate's experience.
Role Relevance (10%)	100% if the job title contains a keyword for the target role.
Location Match (10%)	100% if the preferred city (or its alias) appears in the job location; otherwise 25%.
Work Mode Match (5%)	100% for an exact match, 75% for remote/hybrid partial matches, otherwise 25%.
Eligibility Rule	Experience met, at least 50% core skill match and a relevant job title.
Fit Category	Excellent ≥ 80, Good ≥ 65, Potential ≥ 50, Low ≥ 30, otherwise Very Low.

These rules make every score transparent and reproducible.

Notebook and Web Dashboard Comparison

Evaluation Metric	Notebook (v1)	Web Dashboard (v2)
Scoring method	Row-by-row loop (iterrows)	Vectorised with NumPy
Scoring time per candidate	Python loop over all 23,201 rows	Well under one second
Skill-gap ranking	Most frequent missing skills	Ranked by extra eligible jobs unlocked
Saving backend files	Failed (reset_index error)	Working export
User interface	Notebook outputs	Six-tab interactive dashboard

3.3.5 Overall Discussion

The experiments demonstrate that careful data cleaning and well-chosen visualizations can describe a large job market clearly, and that a transparent scoring engine can give each candidate personalised and explainable guidance. The what-if analysis turns skill recommendations into measurable outcomes, and the AI advisor presents the results in natural language. However, the quality of the output depends on the completeness of the dataset: salary is disclosed in only 11.9% of postings, the skill taxonomy covers only listed skills, and role relevance depends on job-title keywords.

3.4 Conclusion and Future Scope

3.4.1 Conclusion

The project "AI-Assisted Indian Tech Job Market Analysis and Candidate Matching Dashboard" demonstrates the use of data analytics, data visualization and Artificial Intelligence to make the Indian technology job market easier to understand.

The system cleans and standardises a dataset of 23,201 job postings, visualises the market by role, city, work mode, experience, fresher-friendliness, salary and skill demand, ranks roles with an opportunity score, and evaluates a candidate against every posting with clear explanations. Using Python, pandas, NumPy, Matplotlib, LangChain, LangGraph, Google Gemini, FastAPI and Chart.js, the application converts raw postings into meaningful charts and natural-language guidance.

The experiments showed that Data Scientist is the strongest role with 6,455 postings and the highest disclosed salary, that the demonstration candidate was eligible for 46 jobs with a best match of 96.25%, and that learning Power BI and Data Analysis would increase the eligible jobs to 212.

Overall, the project provides a practical application of data analytics and visualization to career guidance. It reduces the effort involved in reading thousands of postings and helps job seekers make informed, data-driven decisions.

3.4.2 Future Scope

The project can be further enhanced through the following improvements:

1. Live Data Collection: Collect postings regularly and add time-series charts showing how demand, skills and salaries change over time.

2. Resume Parsing: Extract skills and experience automatically from an uploaded resume.

3. Semantic Skill Matching: Use text embeddings so that related skills receive partial credit and new skills are recognised automatically.

4. Learned Scoring Weights: Learn the score weights from application outcomes if such data becomes available.

5. Salary Prediction: Estimate salaries for postings that do not disclose them, with clear uncertainty ranges.

6. Geographic Visualization: Show job demand on an interactive map of India and group nearby cities into regions.

7. Learning Roadmaps: Link recommended skills to courses and produce a personalised learning plan.

8. Cloud Deployment: Deploy the dashboard online with user accounts so that profiles and progress can be saved.

In the future, the system can be developed into a complete, continuously updated career-guidance platform that supports data-driven decisions for students and professionals.

References

The following books, research papers, frameworks and documentation were referred to during the development of the project “AI-Assisted Indian Tech Job Market Analysis and Candidate Matching Dashboard.”

1. Tufte, E. R. (2001). The Visual Display of Quantitative Information (2nd ed.). Graphics Press.

2. Munzner, T. (2014). Visualization Analysis and Design. CRC Press. https://doi.org/10.1201/b17511

3. McKinney, W. (2022). Python for Data Analysis (3rd ed.). O'Reilly Media. https://wesmckinney.com/book/

4. Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

5. Harris, C. R., et al. (2020). Array Programming with NumPy. Nature, 585, 357–362. https://doi.org/10.1038/s41586-020-2649-2

6. pandas Development Team. pandas Documentation. https://pandas.pydata.org/docs/

7. Chart.js Contributors. Chart.js Documentation. https://www.chartjs.org/docs/latest/

8. FastAPI. FastAPI Documentation. https://fastapi.tiangolo.com/

9. LangChain. LangChain Documentation. https://python.langchain.com/docs/

10. LangChain. LangGraph Documentation. https://langchain-ai.github.io/langgraph/

11. Google. Gemini API Documentation. https://ai.google.dev/gemini-api/docs

12. Python Software Foundation. Python 3 Documentation. https://docs.python.org/3/